Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS344 - Computer Graphics

Copyright ©2026 by FJ Redelinghuys. All rights reserved.

Practical 1 Specification: Mathematical Programming

Release Date: 16-02-2026 at 06:00

Due Date: 27-02-2026 at 23:59

Total Marks: 97

Contents

1 General Instructions **3**

2 Overview **3**

3 Your Task: **4**

3.1 Object management functions 4

3.1.1 Vector 4

3.1.2 Matrix 5

3.2 Linear algebra functions 6

3.3 Shapes 8

3.3.1 Shape class 8

3.3.2 Triangle class 8

3.3.2.1 Members 8

3.3.2.2 Functions 9

3.3.3 Square class 9

3.3.3.1 Members 9

3.3.3.2 Functions 9

3.4 Provided functions 10

3.4.1 Vector 10

3.4.2 Matrix 11

3.4.3 Triangle 11

3.4.4 Square 11

4 Implementation Details **11**

5 Template Programming Error **12**

6 Upload Checklist **15**

7 Submission **15**

1 General Instructions

- ***Read the entire assignment thoroughly before you begin coding.***
- This assignment should be completed individually.
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.
- Unless otherwise stated, the usage of additional libraries outside of those indicated in the assignment, will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++11**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

For this practical, you will be implementing matrices, vectors, and shapes in C++. This will be used to programmatically understand fundamental linear algebra concepts needed for computer graphics, as well as a basic understanding of how to use them.

3 Your Task:

You are required to implement all of the provided functions in the `.h` file. Section 3.1 explains the object management functions and Section 3.2 explains all the linear algebra functions. Section 3.4 describes the provided functions that **will** be overwritten.

3.1 Object management functions

This section describes the miscellaneous functions that need to be implemented.

3.1.1 Vector

This class represents a vector[1] in a possible N-D space. The size of the vector is specified as an integer template argument.

- `Vector()`
  - This constructor should initialise the array using the template argument.
  - It is at your discretion with what value the elements in the vector should be initialised to.
  - It can be assumed that valid template arguments will be used.
- `Vector(std::initializer_list<float>)`
  - This constructor should initialise the array using the template argument.
  - The array should be populated using the initialiser list.
    - If the initialiser list contains more elements than the array can contain, take the first n value.
    - If the initialiser list contains fewer elements than the array can contain, populate the remaining open spaces in the array with a value of your discretion.
  - *Hint: The following page may be useful,* ***https://www.geeksforgeeks.org/cpp/std-initiali list-in-cpp-11/***
- `Vector(float*)`
  - This constructor should initialise the array using the template argument, and the passed-in parameter.
  - Make use of shallow copies for the array.
  - It can be assumed that the input to this constructor will be valid.
- `Vector(const Matrix<n, 1>&)`
  - This constructor should create a vector from the passed-in matrix.
- `~Vector()`

[1] Please see `https://en.wikipedia.org/wiki/Vector_(mathematics_and_physics)` if you are unsure what a vector is.

  - This destructor should deallocate any dynamically allocated memory.

- `Vector(const Vector<n>&)`
  - This copy constructor should create a deep copy of the passed-in parameter.

- `operator Matrix<n,1>() const;`
  - This is a conversion operator[2] which will return an $N \times 1$ Matrix, populated with the values in the vector.

- `Vector<n>& operator=(const Vector<n>&)`
  - This is an assignment operator[3] and should follow all the conventions of an assignment operator as was taught in COS110.

- `int getN() const`
  - Getter for the size of the vector.

3.1.2 Matrix

This class represents a row-major[4] 2-D matrix of size $N \times M$. The dimensions of the vector is passed as integer template arguments, where the first parameter represents N and the second M.

- `Matrix()`
  - The constructor should initialise the array such that it forms a $N \times M$ matrix, populated with values of your choice.
  - It can be assumed that valid template arguments will be used.

- `Matrix(float**)`
  - The constructor should initialise the array such that it forms a $N \times M$ matrix, populated with values contained in the passed-in parameter.
  - It can be assumed that the parameters will be valid.
  - Use shallow copy to initialize the array member variable.

- `Matrix(const Matrix<n,m> &)`
  - This copy constructor should be used to create a deep copy of the passed-in parameter.

- `virtual ~Matrix()`
  - This virtual destructor should deallocate any dynamically allocated memory.

[2] See option 1 given in the table on the page: `https://en.cppreference.com/w/cpp/language/cast_operator` if you can't remember what a conversion operator is.

[3] Please see `https://en.cppreference.com/w/cpp/language/copy_assignment` if you can't remember what an assignment operator is.

[4] See `https://en.wikipedia.org/wiki/Row-_and_column-major_order` for more information.

- `int getN() const`
  - Getter for the `N` template variable.
- `int getM() const`
  - Getter for the `M` template variable.
- `Matrix<n,m>& operator=(const Matrix<n,m>&)`
  - This is an assignment operator[5] and should follow all the conventions of an assignment operator as was taught in COS110.

3.2 Linear algebra functions

This section describes the linear algebra functions you will be implementing. Assume the following matrices, vectors, and scalars[6] have been defined:
Matrices: $A \in \mathbb{R}^{n \times m}$ and $B \in \mathbb{R}^{m \times r}$
Vectors: $v, w, t \in \mathbb{R}^n$
Scalars: $s \in \mathbb{R}$
Your task will be to implement the functions as described in Table 1.

Take note of the following:

1. For the unit vector function, if the magnitude of the vector is 0, then throw:

```
"Invalid unit vector"
```

2. For the cross product function, the cross product function has been specialised to only work on vector's of size 3.
3. For the magnitude of the vector, use the Euclidean norm[7] for your calculations.
4. For the matrix multiplication function, your template preamble will need to look as follows:

```
template<int n, int m>
template<int a>
```

5. If the determinant of a matrix cannot be calculated, throw the following error:

```
    "Matrix is not square"
```

6. *Hint: Create a global submatrix function that creates a smaller matrix using the original matrix.*

[5] Please see `https://en.cppreference.com/w/cpp/language/copy_assignment` if you can't remember what an assignment operator is.
[6] Please see `https://en.wikipedia.org/wiki/Scalar_(mathematics)` if you are unsure what a scalar is.
[7] Please see `https://en.wikipedia.org/wiki/Norm_(mathematics)` for more information.

Mathematical notation	Function	Description
$v + w$	Vector::operator+	Returns a new vector, which is the result of the vector addition performed on the `this` vector (v) and the passed-in vector (w).
$v - w$	Vector::operator-	Returns a new vector, which is the result of the vector subtraction performed on the `this` vector (v) and the passed-in vector (w).
$v \times s$	Vector::operator*	Returns a new vector which is the result of vector scaling between the `this` vector (v) and the passed-in scalar (s).
$v \cdot w$	Vector::operator*	Returns the dot product between the `this` vector (v) and the passed-in vector (w).
$\|\|v\|\|$	Vector::magnitude	Returns the magnitude of the `this` vector (v). See point 3.
$v \times w$, if $v, w \in \mathbb{R}^3$	Vector::crossProduct	Returns the cross-product between the `this` vector (v) and the passed-in vector (w). See point 2.
$\hat{v}$	Vector::unitVector	Returns the unit vector of the `this` vector (v). See point 1.
$A \times B$	Matrix::operator*	Returns a new matrix, that is the result of the matrix multiplication performed on the `this` matrix (A) and the passed-in matrix (B). See point 4.
$A \times s$	Matrix::operator*	Returns a new matrix, that is the result of scaling the `this` matrix (A) with the passed-in scalar (s).
$A + B$	Matrix::operator+	Returns a new matrix, that is the result of the element wise summation of the `this` matrix (A) with the passed-in matrix (B).
A^T	Matrix::operator~	Returns a new matrix, that is the transpose of the `this` matrix (A).
$det(A)$	Matrix::determinant	This function should return the determinant of the `this` matrix (A). See Point 5.

Table 1: Table explaining all the linear algebra functions that need to be implemented

3.3 Shapes

The shapes hierarchy is required to demonstrate how the above matrix and vector classes can be used to create and manipulate shapes.

3.3.1 Shape class

The shape class has the following pure virtual functions:

- `operator*=`
  - This function takes in a matrix and performs matrix multiplication on each of the vectors that make up the shape.
  - This function should return the modified `this` object.
- `operator*`
  - This function takes in a matrix and performs matrix multiplication on each of the vectors that make up the shape.
  - Note the function should apply the matrix to a copy of the `this` object, and return this copy.
- `getPoints`
  - This function will return an array containing a linearised version of the shape, where each component of each shape of the vectors comprising the shape is added to the array.
- `getNumPoints`
  - This function returns the number of elements in the array that is produced by the `getPoints` function.

Note: You will notice some functions in the derived classes have been omitted. Use the function description as given above to implement the functions in the derived classes.

3.3.2 Triangle class

The `Triangle` class represents a triangle shape and publicly inherits from the `Shape` class. It can be assumed that the n, which indicates the dimensionality that the triangle exists in, has the property $n >= 2$.

3.3.2.1 Members

The `Triangle` class has three `Vector<n>` members, which represent the three points that make up the triangle.

3.3.2.2 Functions

The `Triangle` class has the following functions:

- `Triangle(const Vector<n>& p1, const Vector<n>& p2, const Vector<n>& p3)`
  - This is the constructor and should initialise the points with the parameters, in the same order as they were declared (i.e., parameter `p1` is used to initialise member variable `p1`, and so on).
- `Triangle(const Triangle<n>&)`
  - This is a copy constructor and should make a deep copy of the passed-in parameter.
- `getPoints`
  - Expanding on the description in the `Shape` class, this function will first populate the array with the components of `p1`, `p2`, and lastly `p3`.

3.3.3 Square class

The `Square` class represents a triangle shape and publicly inherits from the `Shape` class. It can be assumed that the n, which indicates the dimensionality that the square exists in, has the property $n >= 2$.

3.3.3.1 Members

The `Square` class has four `Vector<n>` members, which represent the four points that make up the square. The members are:

- `tl` = Top Left
- `tr` = Top Right
- `br` = Bottom Right
- `bl` = Bottom Left

3.3.3.2 Functions

The `Square` class has the following functions:

- `Square(const Vector<n>& centre, float height, float width)`
  - This constructor must calculate the position of each member variable, and initialise them accordingly.
  - It can be assumed that the `centre` parameter is in the centre of the square.
  - Consult Figure 1 to aid your calculations.

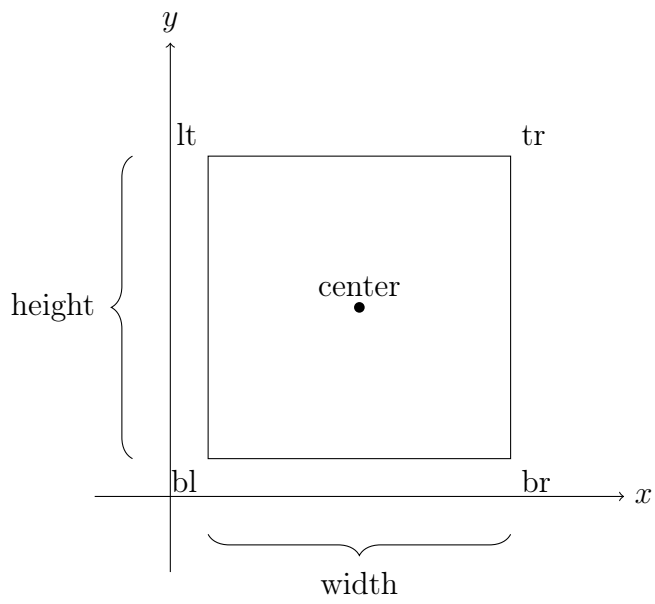

Figure 1: Example figure of the square for the Square constructor

  - Apart from the first and second components, all of the member variables should have the same values for the remaining component as the centre parameter. In other words, if the centre parameter has the value of 5 for the third component, all of the member variables should have the value of 5 for the third component.

- `Square(const Vector<n>& tl, const Vector<n>& br, const Vector<n>& bl)`
  - This constructor should initialise the appropriate member variable with the appropriate parameter.
- `Square(const Square<n>&)`
  - This is a copy constructor and should make a deep copy of the passed-in parameter.
- `getPoints`
  - Expanding on the description in the `Shape` class, this function will first populate the array with the components of `tl`, `tr`, `br`, and lastly `bl`.

3.4 Provided functions

You are provided with the following functions in the `h` files. **Do not** change these functions as they will be overwritten on FitchFork.

3.4.1 Vector

- Subscript operator
  - This can be used to access and set elements in the vector.
- Print function

- This function is used by the marking script to print out the vector.

3.4.2 Matrix

- Subscript operator
  - This can be used to access and set elements in the vector.
- Print function
  - This function is used by the marking script to print out the vector.

3.4.3 Triangle

- Print function
  - This function prints out the coordinates for each of the points in the triangle, using `Vector`'s print function.

3.4.4 Square

- Print function
  - This function prints out the coordinates for each of the points in the triangle, using `Vector`'s print function.

4 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use **C++11**.
- You may only utilize the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- Do not include `using namespace std` in any of the files.
- You may only use the following libraries:
  - IOStream
  - CMath
  - iomanip
  - initializer_list
- You will notice that each submission uses randomly generated matrices, vectors, triangles and squares. This is to allow you to view your output on FitchFork and see where your code failed.
- Ensure you do not have any debugging output, as this can interfere with the marking script.

- The exceptions will not be explicitly tested. They are there to assist you while testing your own code such that you are able to find errors easier.
- When adding helper functions, add them as global functions in the `cpp` files. This implies you will need to use the public interface to access member variables of objects. It is highly recommended that you make use of global helper functions.

5 Template Programming Error

Due to the use of template arguments to dictate the sizes of the vectors and matrices, you are effectively getting the compiler to create different classes for different size vectors and matrices. This is a simple example of template programming. The use of template programming can greatly speed up your program and ensure that you use the correct sizes for different functions.

While coding your assignment, you may encounter the following error:

```
    fatal error: template instantiation depth exceeds maximum of 900 (use
        '-ftemplate-depth=' to increase the maximum)
```

This happens due to your program exceeding the compiler's recursive depth limit. To illustrate this, an example is provided below:

```
#include <iostream>

using namespace std;

template <int n>
class Counter{
    public:
        Counter<n-1> countDown();

};

template <int n>
Counter<n-1> Counter<n>::countDown()
{
    cout << n << endl;
    Counter<n-1> next;
    next.countDown();
    return next;
}

int main(){
    Counter<10> counter;
    counter.countDown();
}
```

As can be seen in this code example, a `Counter` class is created for $n = 10$, which needs to create a `Counter` class for $n = 9$, which in turn needs to make a `Counter` class $n = 8$. This is repeated all the way to $n = -889$, where the compile recursive depth limit was reached. To avoid this, a recursive

base case is needed. The easiest way to do this is to specialise the `Counter` class for the base case, which is $n = 0$. This can be done as follows:

```
#include <iostream>

using namespace std;

template <int n>
class Counter{
    public:
        Counter<n-1> countDown();

};

template <int n>
Counter<n-1> Counter<n>::countDown()
{
    cout << n << endl;
    Counter<n-1> next;
    next.countDown();
    return next;
}

template<>
Counter<-1> Counter<0>::countDown(){
    cout << "0 - blast-off" << endl;
    Counter<-1> next;
    return next;
}

int main(){
    Counter<10> counter;
    counter.countDown();
}
```

The code will now correctly compile and all the `Counter` classes will be created at compile time and available for immediate use at run time. Note that this is a trivial example of the power of compile-time programming, and more details on this subject are left for `COS782`.

Hint: you may find this handy when you are creating matrices of size $n - 1$.

6 Upload Checklist

The following C++ files should be in a zip archive named uXXXXXXXX.zip where XXXXXXXX is your student number:

- `Matrix.h`
- `Matrix.cpp`
- `Vector.h`
- `Vector.cpp`
- `Triangle.h`
- `Triangle.cpp`
- `Square.h`
- `Square.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

Skeleton files have been provided for you with correct linking. Do not change the linking provided, as this may lead to compilation errors. Also, do not change any of the provided functions. This is used by FitchFork to print out your matrices and vectors.

7 Submission

You need to submit your source files on the FitchFork website (https://ff.cs.up.ac.za/). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++11 -g *.cpp -o main
```

and run with the following command:

```
./main
```

Remember your `h` file will be overwritten, so ensure you do not alter the provided `h` files.
You have 20 submissions and your best mark will be your final mark. Upload your archive to the Practical 1 slot on the FitchFork website. Submit your work before the deadline. **No late submissions will be accepted!**